

Now, we add one more critical piece to virtualization puzzle: persistent storage.

A persistent storage device such as HDD/SSD stores information permanently.

CRUX: HOW TO MANAGE A PERSISTENT DEVICE

How should the OS manage a persistent device? What are the APIs? What are the important aspects of the implementation?

Files & Directories

Two key abstractions have been developed over time.

1. File:

It is simply a linear array of bytes, each of which you can write or read.

Each file has some kind of low-level name, a number, called inode number (i-number).

In most systems, the OS does not know much about the structure of the file (e.g., whether it is a picture, or a text file, or C code); rather, the responsibility of the file system is simply to store such data persistently on disk and make sure that when you request the data again, you get what you put there in the first place. Doing so is not as simple as it seems!

2. Directory:

It also has an inode number.

It contains a list of

(user-readable name, low-level name) pairs.

Each entry in a directory is a file or other directories.

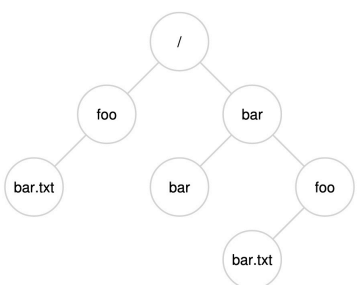


Figure 39.1: An Example Directory Tree

Through this users are able to build an arbitrary directory tree/hierarchy

Directories and files can have the same name as long as they are in different locations in the file-system tree (e.g., there are two files named `bar.txt` in the figure, `/foo/bar.txt` and `/bar/foo/bar.txt`).

You may also notice that the file name in this example often has two parts: `bar` and `txt`, separated by a period. The first part is an arbitrary name, whereas the second part of the file name is usually used to indicate the **type** of the file, e.g., whether it is C code (e.g., `.c`), or an image (e.g., `.jpg`), or a music file (e.g., `.mp3`). However, this is usually just a **convention**: there is usually no enforcement that the data contained in a file named `main.c` is indeed C source code.

Thus, we can see one great thing provided by the file system: a convenient way to **name** all the files we are interested in. Names are important in systems as the first step to accessing any resource is being able to name it. In UNIX systems, the file system thus provides a unified way to access files on disk, USB stick, CD-ROM, many other devices, and in fact many other things, all located under the single directory tree.

Creating Files

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC,  
              S_IRUSR|S_IWUSR);
```

`open()` takes a number of flags:

`O_CREAT`: Create the file if it doesn't exist

`O_WRONLY`: Ensures file can only be written to

`O_TRUNC`: Truncates file size to 0 bytes

The third parameter specifies permissions, in this case making it readable & writeable by owner.

It returns a file descriptor, which is an integer private per process managed by OS.

As such, an array is kept in the `proc` structure

```
struct proc {  
    ...  
    struct file *ofile[NOFILE]; // Open files  
    ...  
};
```


It is a simple array with a maximum of `NOFILE` open files indexed by the file descriptor.

Each array entry is a pointer to a struct file.

Reading And Writing Files

Example of using `strace` to figure out what `cat` is doing.

```
prompt> strace cat foo
```

```
...
```

```
open("foo", O_RDONLY|O_LARGEFILE)           = 3
```

```
read(3, "hello\n", 4096)                     = 6
```

```
write(1, "hello\n", 6)                       = 6
```

```
hello
```

```
read(3, "", 4096)                           = 0
```

```
close(3)                                     = 0
```

```
...
```

```
prompt>
```

The first thing that `cat` does is open the file for reading. A couple of things we should note about this; first, that the file is only opened for reading (not writing), as indicated by the `O_RDONLY` flag; second, that the 64-bit offset is used (`O_LARGEFILE`); third, that the call to `open()` succeeds and returns a file descriptor, which has the value of 3.

Why does the first call to `open()` return 3, not 0 or perhaps 1 as you might expect? As it turns out, each running process already has three files open, standard input (which the process can read to receive input), standard output (which the process can write to in order to dump information to the screen), and standard error (which the process can write error messages to). These are represented by file descriptors 0, 1, and 2, respectively. Thus, when you first open another file (as `cat` does above), it will almost certainly be file descriptor 3.

After the open succeeds, `cat` uses the `read()` system call to repeatedly read some bytes from a file. The first argument to `read()` is the file descriptor, thus telling the file system which file to read; a process can of course have multiple files open at once, and thus the descriptor enables the operating system to know which file a particular read refers to. The second argument points to a buffer where the result of the `read()` will be placed; in the system-call trace above, `strace` shows the results of the read in this spot ("hello"). The third argument is the size of the buffer, which in this case is 4 KB. The call to `read()` returns successfully as well, here returning the number of bytes it read (6, which includes 5 for the letters in the word "hello" and one for an end-of-line marker).

At this point, you see another interesting result of the `strace`: a single call to the `write()` system call, to the file descriptor 1. As we mentioned above, this descriptor is known as the standard output, and thus is used to write the word "hello" to the screen as the program `cat` is meant to do. But does it call `write()` directly? Maybe (if it is highly optimized). But if not, what `cat` might do is call the library routine `printf()`; internally, `printf()` figures out all the formatting details passed to it, and eventually writes to standard output to print the results to the screen.

The `cat` program then tries to read more from the file, but since there are no bytes left in the file, the `read()` returns 0 and the program knows that this means it has read the entire file. Thus, the program calls `close()` to indicate that it is done with the file "foo", passing in the corresponding file descriptor. The file is thus closed, and the reading of it thus complete.

Writing a file is accomplished via a similar set of steps. First, a file is opened for writing, then the `write()` system call is called, perhaps repeatedly for larger files, and then `close()`. Use `strace` to trace writes to a file, perhaps of a program you wrote yourself, or by tracing the `dd` utility, e.g., `dd if=foo of=bar`.

ASIDE: DATA STRUCTURE — THE OPEN FILE TABLE

Each process maintains an array of file descriptors, each of which refers to an entry in the system-wide **open file table**. Each entry in this table tracks which underlying file the descriptor refers to, the current offset, and other relevant details such as whether the file is readable or writable.

Reading And Writing, But Not Sequentially

Thus far, we have read from/written to a file from beginning to end.

We use `lseek()` to read from/write to random offsets within a document.

```
off_t lseek(int fildes, off_t offset, int whence);
```

`offset`: Positions the file offset to a particular location

`whence`: Determines how seek is performed

If `whence` is `SEEK_SET`, start from beginning of file offset bytes from start

If `whence` is `SEEK_CUR`, the offset is set to its current location plus offset bytes.

If `whence` is `SEEK_END`, the offset is set to the size of the file plus offset bytes.

As you can tell from this description, for each file a process opens, the OS tracks a "current" offset, which determines where the next read or write will begin reading from or writing to within the file. Thus, part of the abstraction of an open file is that it has a current offset, which is updated in one of two ways. The first is when a read or write of N bytes takes place, N is added to the current offset; thus each read or write *implicitly* updates the offset. The second is *explicitly* with `lseek`, which changes the offset as specified above.

The offset, as you might have guessed, is kept in that `struct file` we saw earlier, as referenced from the `struct proc`. Here is a (simplified) xv6 definition of the structure:

```
struct file {
    int ref;
    char readable;
    char writable;
    struct inode *ip;
    uint off;
};
```

The OS can use this to determine whether file is readable or writable or both.

Which underlying file it refers to (as pointed to by `struct inode` pointer), current offset (`off`).

There is also a reference count (`ref`).

These file structures represent all of the currently opened files in the system. Together they are referred to as a open file table.

The xv6 kernel just keeps these as an array with one lock for the entire table:

```
struct {
    struct spinlock lock;
    struct file file[NFILE];
} ftable;
```

Examples,

1. Lets track a process that opens a file of size 300 bytes and reads it by calling the `read()` system call repeatedly, each time reading 100

bytes.

System Calls	Return Code	Current Offset
fd = open("file", O_RDONLY);	3	0
read(fd, buffer, 100);	100	100
read(fd, buffer, 100);	100	200
read(fd, buffer, 100);	100	300
read(fd, buffer, 100);	0	300
close(fd);	0	—

Interesting points:

- Current offset is initialized to 0 when file is opened.
- It is incremented with each read()
- At the end, an attempted read() past end of file returns 0.

2. Lets trace a process that opens the same file twice & issues reads to them.

OFT: open file table

System Calls	Return Code	OFT[10] Current Offset	OFT[11] Current Offset
fd1 = open("file", O_RDONLY);	3	0	—
fd2 = open("file", O_RDONLY);	4	0	0
read(fd1, buffer1, 100);	100	100	0
read(fd2, buffer2, 100);	100	100	100
close(fd1);	0	—	100
close(fd2);	0	—	—

In this, two file descriptors are allocated (3 & 4), and each refers to a different entry in the open file table.

Each current offset is update independently.

3. A process uses lseek() to reposition the current offset before reading.

System Calls	Return Code	Current Offset
<code>fd = open("file", O_RDONLY);</code>	3	0
<code>lseek(fd, 200, SEEK_SET);</code>	200	200
<code>read(fd, buffer, 50);</code>	50	250
<code>close(fd);</code>	0	—

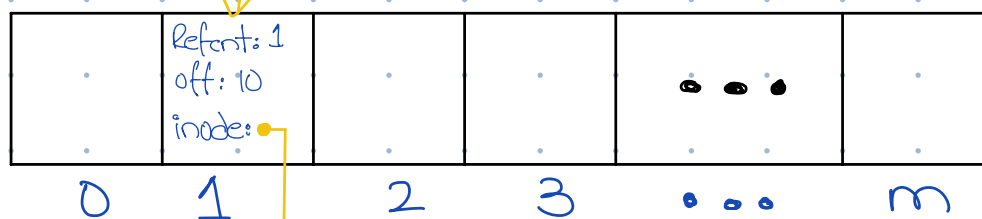
Here, `lseek()` call first sets current offset to 200. The `read()` reads next 50 bytes & updates the offset.

Each entry is pointer to file struct



process' file descriptor array

An array of file struct



kernel's open file table

Inode #1000
file.txt

ASIDE: CALLING `lseek()` DOES NOT PERFORM A DISK SEEK

The poorly-named system call `lseek()` confuses many a student trying to understand disks and how the file systems atop them work. Do not confuse the two! The `lseek()` call simply changes a variable in OS memory that tracks, for a particular process, at which offset its next read or write will start. A disk seek occurs when a read or write issued to the disk is not on the same track as the last read or write, and thus necessitates a head movement. Making this even more confusing is the fact that calling `lseek()` to read or write from/to random parts of a file, and then reading/writing to those random parts, will indeed lead to more disk seeks. Thus, calling `lseek()` can lead to a seek in an upcoming read or write, but absolutely does not cause any disk I/O to occur itself.

Shared File Table Entries: fork() And dup()

In many cases, the mapping of file descriptors to an entry in the open file table is one-to-one mapping.

However, there are few cases where an entry in the open file table is shared.

First is when a parent process creates a child as:

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
               (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

Figure 39.2: Shared Parent/Child File Table Entries (fork-seek.c)

When we run this, we get output:

```
prompt> ./fork-seek
child: offset 10
parent: offset 10
prompt>
```

Figure 39.3 shows the relationships that connects each process' private descriptor array, the shared open file table entry, and reference from it to underlying file-system inode.

We make use of reference count here. When a file table entry is shared, its reference count is incremented; only when both processes close the file (or exit) will the entry be removed.

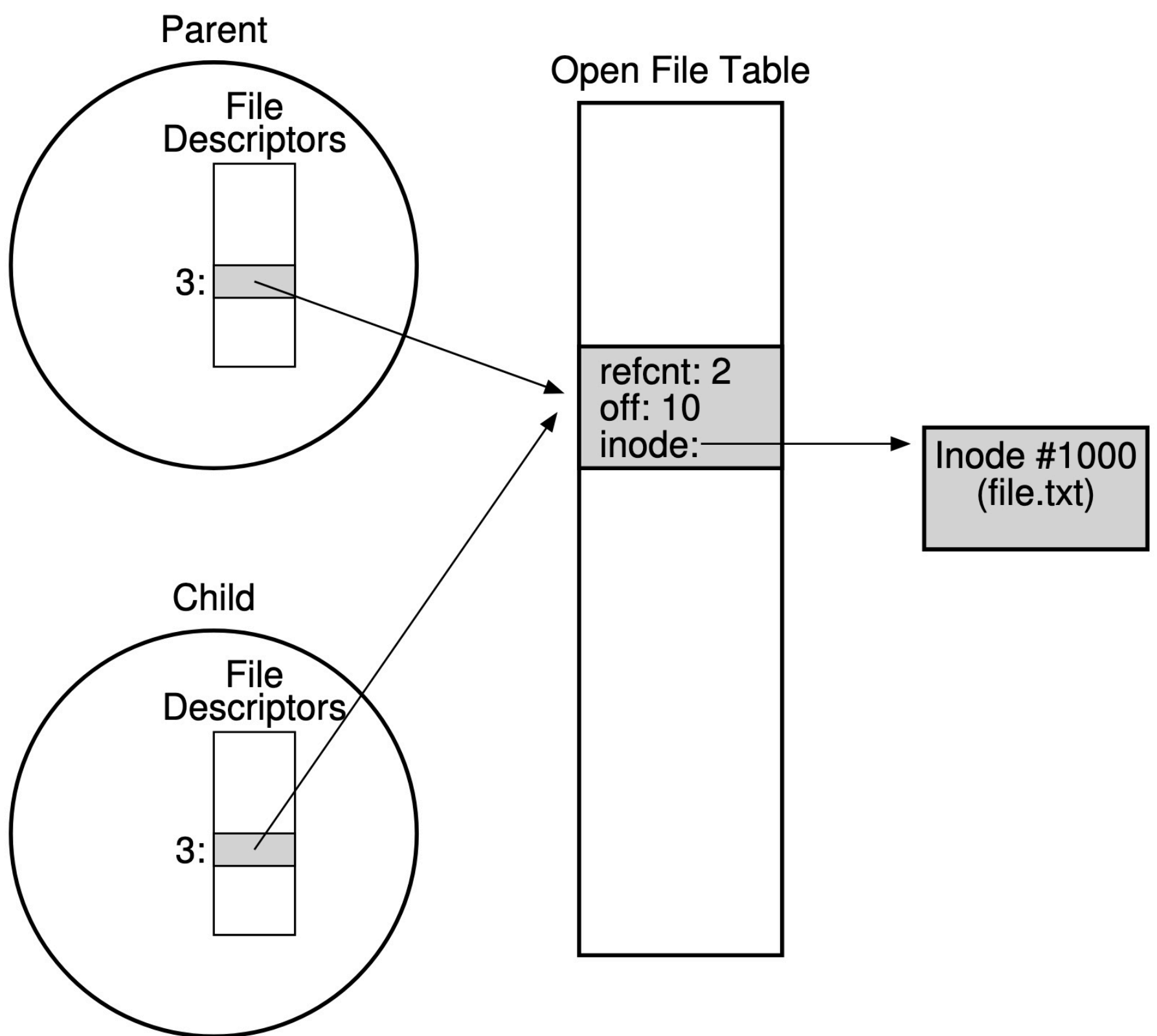


Figure 39.3: Processes Sharing An Open File Table Entry

Second, the `dup()` call allows a process to create a new file descriptor that refers to the same underlying open file.

```
int main(int argc, char *argv[]) {
    int fd = open("README", O_RDONLY);
    assert(fd >= 0);
    int fd2 = dup(fd);
    // now fd and fd2 can be used interchangeably
    return 0;
}
```

Figure 39.4: Shared File Table Entry With `dup()` (`dup.c`)

Writing Immediately With `fsync()`

Most times when a program calls `write()`, it is just telling the file system: please write this data to persistent storage at some point in the future.

The filesystem will buffer such writes in memory for some time.

However, in DBMS, the development of a correct recovery protocol requires the ability to force write to disk.

Thus, when a process calls `fsync(int fd)` for a particular file descriptor, the file system responds by forcing all dirty data to disk.

The `fsync()` routine returns once all of these writes are complete.

Here is a simple example of how to use `fsync()`. The code opens the file `foo`, writes a single chunk of data to it, and then calls `fsync()` to ensure the writes are forced immediately to disk. Once the `fsync()` returns, the application can safely move on, knowing that the data has been persisted (if `fsync()` is correctly implemented, that is).

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC,  
              S_IRUSR|S_IWUSR);  
assert(fd > -1);  
int rc = write(fd, buffer, size);  
assert(rc == size);  
rc = fsync(fd);  
assert(rc == 0);
```

Interestingly, this sequence does not guarantee everything that you might expect; in some cases, you also need to `fsync()` the directory that contains the file `foo`. Adding this step ensures not only that the file itself is on disk, but that the file, if newly created, also is durably a part of the directory.

Directory's list of pairs must also be updated.

allows you access file data as if they were in memory.

ASIDE: `mmap()` AND PERSISTENT MEMORY (Guest Aside by Terence Kelly)

Memory mapping is an alternative way to access persistent data in files. The `mmap()` system call creates a correspondence between byte offsets in a file and virtual addresses in the calling process; the former is called the **backing file** and the latter its **in-memory image**. The process can then access the backing file using CPU instructions (i.e., loads and stores) to the in-memory image.

By combining the persistence of files with the access semantics of memory, file-backed memory mappings support a software abstraction called **persistent memory**. The persistent memory style of programming can streamline applications by eliminating translation between different data formats for memory and storage [K19].

```
1 p = mmap(NULL, file_size, PROT_READ|PROT_WRITE,
2         MAP_SHARED, fd, 0);
3 assert(p != MAP_FAILED);
4 for (int i = 1; i < argc; i++)
5     if (strcmp(argv[i], "pop") == 0) // pop
6         if (p->n > 0) // stack not empty
7             printf("%d\n", p->stack[--p->n]);
8     } else { // push
9         if (sizeof(pstack_t) + (1 + p->n) * sizeof(int)
10            <= file_size) // stack not full
11             p->stack[p->n++] = atoi(argv[i]);
12     }
```

The program `pstack.c` (included on the OSTEP code github repo, with a snippet shown above) stores a persistent stack in file `ps.img`, which begins life as a bag of zeros, e.g., created on the command line via the `truncate` or `dd` utility. The file contains a count of the size of the stack and an array of integers holding stack contents. After `mmap()`-ing the backing file we can access the stack using C pointers to the in-memory image, e.g., `p->n` accesses the number of items on the stack, and `p->stack` the array of integers. Because the stack is persistent, data push'd by one invocation of `pstack` can be pop'd by the next.

A crash, e.g., between the increment and the assignment of the push, could leave our persistent stack in an inconsistent state. Applications prevent such damage by using mechanisms that update persistent memory atomically with respect to failure [K20].

Renaming files

```
prompt> mv foo bar
```

Using `strace` reveals that `mv` uses the system call

`rename(char*old, char*new)`, which takes precisely

two arguments: the original name of the file (`old`) and the new name (`new`).

`rename()` call is implemented as an atomic call w.r.t. system crashes i.e. if a system crashes during renaming, the file will either be named the old name or new name, and no odd in-b/w state can arise.

An interesting application is:

Let's be a little more specific here. Imagine that you are using a file editor (e.g., `emacs`), and you insert a line into the middle of a file. The file's name, for the example, is `foo.txt`. The way the editor might update the file to guarantee that the new file has the original contents plus the line inserted is as follows (ignoring error-checking for simplicity):

```
int fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC,  
              S_IRUSR|S_IWUSR);  
write(fd, buffer, size); // write out new version of file  
fsync(fd);  
close(fd);  
rename("foo.txt.tmp", "foo.txt");
```

What the editor does in this example is simple: write out the new version of the file under a temporary name (`foo.txt.tmp`), force it to disk with `fsync()`, and then, when the application is certain the new file metadata and contents are on the disk, rename the temporary file to the original file's name. This last step atomically swaps the new file into place, while concurrently deleting the old version of the file, and thus an atomic file update is achieved.

Getting Information About Files

Beyond file access, we expect the file system to keep a fair amount of information about each file it is storing. We generally call such data about files **metadata**. To see the metadata for a certain file, we can use the `stat()` or `fstat()` system calls. These calls take a pathname (or file descriptor) to a file and fill in a `stat` structure as seen in Figure 39.5.

You can see that there is a lot of information kept about each file, including its size (in bytes), its low-level name (i.e., inode number), some ownership information, and some information about when the file was accessed or modified, among other things. To see this information, you can use the command line tool `stat`. In this example, we first create a file (called `file`) and then use the `stat` command line tool to learn some things about the file.

```
struct stat {
    dev_t      st_dev;        // ID of device containing file
    ino_t      st_ino;        // inode number
    mode_t     st_mode;       // protection
    nlink_t    st_nlink;      // number of hard links
    uid_t      st_uid;        // user ID of owner
    gid_t      st_gid;        // group ID of owner
    dev_t      st_rdev;       // device ID (if special file)
    off_t      st_size;       // total size, in bytes
    blksize_t  st_blksize;    // blocksize for filesystem I/O
    blkcnt_t   st_blocks;     // number of blocks allocated
    time_t     st_atime;      // time of last access
    time_t     st_mtime;      // time of last modification
    time_t     st_ctime;      // time of last status change
};
```

Figure 39.5: The **stat** structure.

Here is the output on Linux:

```
prompt> echo hello > file
prompt> stat file
  File: 'file'
  Size: 6    Blocks: 8    IO Block: 4096    regular file
Device: 811h/2065d Inode: 67158084    Links: 1
Access: (0640/-rw-r-----)  Uid: (30686/remzi)
  Gid: (30686/remzi)
Access: 2011-05-03 15:50:20.157594748 -0500
Modify: 2011-05-03 15:50:20.157594748 -0500
Change: 2011-05-03 15:50:20.157594748 -0500
```

Each file system usually keeps this type of information in a structure called an **inode**¹. We'll be learning a lot more about inodes when we talk about file system implementation. For now, you should just think of an inode as a persistent data structure kept by the file system that has information like we see above inside of it. All inodes reside on disk.

information like we see above inside of it. All inodes reside on disk; a copy of active ones are usually cached in memory to speed up access.

Removing Files

Run program `rm`.

What system call `rm` uses:

```
prompt> strace rm foo
```

```
...
```

```
unlink("foo")                                = 0
```

```
...
```

Making Directories

You can never write to a directory directly. As format of the directory is considered file system metadata, the file system considers itself responsible for integrity of directory data.

Thus, you can update a directory indirectly by creating files in it.

Create it as:

```
prompt> strace mkdir foo
```

```
...
```

```
mkdir("foo", 0777)                                = 0
```

```
...
```

```
prompt>
```

when it is first created, it is considered empty, but it contains `"."` & `".."`

Reading Directories

Instead of just opening a directory as if it were a file, we instead use a new set of calls. Below is an example program that prints the contents of a directory. The program uses three calls, `opendir()`, `readdir()`, and `closedir()`, to get the job done, and you can see how simple the interface is; we just use a simple loop to read one directory entry at a time, and print out the name and inode number of each file in the directory.

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir(".");
    assert(dp != NULL);
    struct dirent *d;
    while ((d = readdir(dp)) != NULL) {
        printf("%lu %s\n", (unsigned long) d->d_ino,
               d->d_name);
    }
    closedir(dp);
    return 0;
}
```

The declaration below shows the information available within each directory entry in the `struct dirent` data structure:

```
struct dirent {
    char          d_name[256]; // filename
    ino_t         d_ino;       // inode number
    off_t         d_off;       // offset to the next dirent
    unsigned short d_reclen;    // length of this record
    unsigned char  d_type;      // type of file
};
```

Deleting Directories

Use `rmdir()` to delete empty directories.

Hard Links

We now come back to the mystery of why removing a file is performed via `unlink()`, by understanding a new way to make an entry in the file system tree, through a system call known as `link()`. The `link()` system call takes two arguments, an old pathname and a new one; when you “link” a new file name to an old one, you essentially create another way to refer to the same file. The command-line program `ln` is used to do this, as we see in this example:


```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2
prompt> cat file2
hello
```

The way `link()` works is that it simply creates another name in the directory you are creating the link to, and refers it to the *same* inode number (i.e., low-level name) of the original file. The file is not copied in any way; rather, you now just have two human-readable names (`file` and `file2`) that both refer to the same file. We can even see this in the directory itself, by printing out the inode number of each file:

```
prompt> ls -li file file2
67158084 file
67158084 file2
prompt>
```

By passing the `-li` flag to `ls`, it prints out the inode number of each file (as well as the file name). And thus you can see what `link` really has done: just make a new reference to the same exact inode number (67158084 in this example).

By now you might be starting to see why `unlink()` is called `unlink()`. When you create a file, you are really doing *two* things. First, you are making a structure (the inode) that will track virtually all relevant information about the file, including its size, where its blocks are on disk, and so forth. Second, you are *linking* a human-readable name to that file, and putting that link into a directory.

After creating a hard link to a file, the file system perceives no difference between the original file name (`file`) and the newly created file name (`file2`); indeed, they are both just links to the underlying metadata about the file, which is found in inode number 67158084.

Thus, to remove a file from the file system, we call `unlink()`. In the example above, we could for example *remove the file named `file`*, and still access the file without difficulty:

```
prompt> rm file
removed 'file'
prompt> cat file2
hello
```

The reason this works is because when the file system unlinks `file`, it checks a **reference count** within the inode number. This reference count (sometimes called the **link count**) allows the file system to track how many different file names have been linked to this particular inode. When `unlink()` is called, it removes the “link” between the human-readable

name (the file that is being deleted) to the given inode number, and decrements the reference count; only when the reference count reaches zero does the file system also free the inode and related data blocks, and thus truly “delete” the file.

You can see the reference count of a file using `stat()` of course. Let's see what it is when we create and delete hard links to a file. In this example, we'll create three links to the same file, and then delete them. Watch the link count!

```
prompt> echo hello > file
prompt> stat file
... Inode: 67158084      Links: 1 ...
prompt> ln file file2
prompt> stat file
... Inode: 67158084      Links: 2 ...
prompt> stat file2
... Inode: 67158084      Links: 2 ...
prompt> ln file2 file3
prompt> stat file
... Inode: 67158084      Links: 3 ...
prompt> rm file
prompt> stat file2
... Inode: 67158084      Links: 2 ...
prompt> rm file2
prompt> stat file3
... Inode: 67158084      Links: 1 ...
prompt> rm file3
```

Symbolic Links

There is one other type of link that is really useful, and it is called a **symbolic link** or sometimes a **soft link**. Hard links are somewhat limited: you can't create one to a directory (for fear that you will create a cycle in the directory tree); you can't hard link to files in other disk partitions (because inode numbers are only unique within a particular file system, not across file systems); etc. Thus, a new type of link called the symbolic link.

You can create it as:

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
```

The symbolic is actually a file itself. A stat on symlink reveals all:

```
prompt> stat file
... regular file ...
prompt> stat file2
... symbolic link ...
```

Also does `ls -al`

```
prompt> ls -al
drwxr-x---  2 remzi remzi   29 May  3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May  3 15:14 ../
-rw-r----- 1 remzi remzi    6 May  3 19:10 file
lrwxrwxrwx  1 remzi remzi    4 May  3 19:10 file2 -> file
```

directory
file
soft links

The reason that file2 is 4 bytes is because a symlink is formed by holding the pathname of the linked-to file as data of the link file.

Because of this, we can have dangling reference:

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```


Permission Bits And Access Control Lists

→ raw bytes to user friendly files & directories

This abstraction is notably different from that of CPU & memory, in that files are commonly shared among different users and processes and are not (always) private.

Thus, a more comprehensive mechanism for enabling various degrees of sharing is usually present within file systems.

The first is UNIX permission bits:

```
prompt> ls -l foo.txt
-rw-r--r-- 1 remzi wheel 0 Aug 24 16:29 foo.txt
  owner group other
```

r: file is readable

w: file is writable

x: For regular files, its presence determines whether a program can be run or not.

For directories, it enables a user (group or everyone) to cd, create files therein.

Beyond permissions bits, some file systems, such as the distributed file system known as AFS (discussed in a later chapter), include more sophisticated controls. AFS, for example, does this in the form of an **access control list (ACL)** per directory. Access control lists are a more general and powerful way to represent exactly who can access a given resource. In a file system, this enables a user to create a very specific list of who can and cannot read a set of files, in contrast to the somewhat limited owner/group/everyone model of permissions bits described above.

For example, here are the access controls for a private directory in one author's AFS account, as shown by the `fs listacl` command:

```
prompt> fs listacl private
Access list for private is
Normal rights:
system:administrators rliwka
remzi rliwka
```


The listing shows that both the system administrators and the user `remzi` can lookup, insert, delete, and administer files in this directory, as well as read, write, and lock those files.

To allow someone (in this case, the other author) to access this directory, user `remzi` can just type the following command.

```
prompt> fs setacl private/ andrea rl
```

Making And Mounting A File System

We've now toured the basic interfaces to access files, directories, and certain special types of links. But there is one more topic we should discuss: how to assemble a full directory tree from many underlying file systems. This task is accomplished via first making file systems, and then mounting them to make their contents accessible.

To make a file system, most file systems provide a tool, usually referred to as `mkfs` (pronounced "make fs"), that performs exactly this task. The idea is as follows: give the tool, as input, a device (such as a disk partition, e.g., `/dev/sda1`) and a file system type (e.g., `ext3`), and it simply writes an empty file system, starting with a root directory, onto that disk partition. And `mkfs` said, let there be a file system!

However, once such a file system is created, it needs to be made accessible within the uniform file-system tree. This task is achieved via the `mount` program (which makes the underlying system call `mount()` to do the real work). What `mount` does, quite simply is take an existing directory as a target **mount point** and essentially paste a new file system onto the directory tree at that point.

An example here might be useful. Imagine we have an unmounted `ext3` file system, stored in device partition `/dev/sda1`, that has the following contents: a root directory which contains two sub-directories, `a` and `b`, each of which in turn holds a single file named `foo`. Let's say we wish to mount this file system at the mount point `/home/users`. We would type something like this:

```
prompt> mount -t ext3 /dev/sda1 /home/users
```

If successful, the `mount` would thus make this new file system available. However, note how the new file system is now accessed. To look at the contents of the root directory, we would use `ls` like this:

```
prompt> ls /home/users/  
a b
```


As you can see, the pathname `/home/users/` now refers to the root of the newly-mounted directory. Similarly, we could access directories `a` and `b` with the pathnames `/home/users/a` and `/home/users/b`. Finally, the files named `foo` could be accessed via `/home/users/a/foo` and `/home/users/b/foo`. And thus the beauty of `mount`: instead of having a number of separate file systems, `mount` unifies all file systems into one tree, making naming uniform and convenient.

To see what is mounted on your system, and at which points, simply run the `mount` program. You'll see something like this:

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/sda5 on /tmp type ext3 (rw)
/dev/sda7 on /var/vice/cache type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

This crazy mix shows that a whole number of different file systems, including `ext3` (a standard disk-based file system), the `proc` file system (a file system for accessing information about current processes), `tmpfs` (a file system just for temporary files), and `AFS` (a distributed file system) are all glued together onto this one machine's file-system tree.

TIP: BE WARY OF TOCTTOU

In 1974, McPhee noticed a problem in computer systems. Specifically, McPhee noted that "... if there exists a time interval between a validity-check and the operation connected with that validity-check, [and,] through multitasking, the validity-check variables can deliberately be changed during this time interval, resulting in an invalid operation being performed by the control program." We today call this the **Time Of Check To Time Of Use (TOCTTOU)** problem, and alas, it still can occur.

A simple example, as described by Bishop and Dilger [BD96], shows how a user can trick a more trusted service and thus cause trouble. Imagine, for example, that a mail service runs as `root` (and thus has privilege to access all files on a system). This service appends an incoming message to a user's inbox file as follows. First, it calls `lstat()` to get information about the file, specifically ensuring that it is actually just a regular file owned by the target user, and not a link to another file that the mail server should not be updating. Then, after the check succeeds, the server updates the file with the new message.

Unfortunately, the gap between the check and the update leads to a problem: the attacker (in this case, the user who is receiving the mail, and thus has permissions to access the inbox) switches the inbox file (via a call to `rename()`) to point to a sensitive file such as `/etc/passwd` (which holds information about users and their passwords). If this switch hap-

pens at just the right time (between the check and the access), the server will blithely update the sensitive file with the contents of the mail. The attacker can now write to the sensitive file by sending an email, an escalation in privilege; by updating `/etc/passwd`, the attacker can add an account with `root` privileges and thus gain control of the system.

There are not any simple and great solutions to the TOCTTOU problem [T+08]. One approach is to reduce the number of services that need root privileges to run, which helps. The `O_NOFOLLOW` flag makes it so that `open()` will fail if the target is a symbolic link, thus avoiding attacks that require said links. More radical approaches, such as using a **transactional file system** [H+18], would solve the problem, but there aren't many transactional file systems in wide deployment. Thus, the usual (lame) advice: be careful when you write code that runs with high privileges!